Óptica Vision — Plataforma web para la gestión de una óptica

Raúl Miguel Almarza Robles

CICLO FORMATIVO DESARROLLO DE APLICACIONES WEB

FP Euroformac

Tutor: Eugenio Rodríguez Velasco

8/05/2026

Contenido

1. Resumen 4
2. Introducción 5
    2.1 Presentación del proyecto 5
    2.2 Motivación y origen de la idea 5
    2.3 Contexto (problema detectado o necesidad) 5
    2.4 Objetivos generales y específicos 5
    2.5 Metodología empleada 5
3. Análisis Inicial 6
    3.1 Descripción del problema o necesidad 6
    3.2 Usuarios objetivos / públicos involucrados 6
    3.3 Requisitos del proyecto 6
    3.4 Requisitos funcionales 6
    3.5 Requisitos no funcionales 6
    3.6 Alcance y limitaciones 7
4. Estado del Arte / Tecnologías Existentes 7
    4.1 Soluciones similares existentes 7
    4.2 Comparativa de herramientas 7
    4.3 Justificación tecnológica 7
    4.4 Lenguajes 7
    4.5 Frameworks 7
    4.6 Librerías 8
    4.7 Bases de datos 8
    4.8 Servicios cloud / hosting 8
    4.9 Hardware / infraestructura (si aplica) 8
5. Diseño del Proyecto 8
    5.1 Diseño funcional 8
        Casos de uso 8
    5.2 Diseño técnico 9
        Arquitectura del sistema 9
        Modelo entidad–relación 9
        Diseño lógico de la base de datos 9
        Diseño de interfaz (mockups, wireframes) 9
6. Desarrollo / Implementación 10
    6.1 Entorno de desarrollo 10
    6.2 Configuraciones importantes 10
    6.3 Desarrollo de módulos 10
    6.4 Integraciones externas 10
    6.5 Gestión del código y control de versiones (Git) 11
    6.6 Pruebas realizadas 11

6.7 Pruebas unitarias 11
6.8 Pruebas de integración 11
6.9 Rendimiento 11
7. Seguridad 12
7.1 Identificación de riesgos 12
7.2 Autenticación y autorización 12
7.3 Gestión de sesiones 12
7.4 Cifrado de datos 12
8. Resultados 12
8.1 Presentación del producto final 12
8.2 Capturas de pantalla o diagramas finales 13
8.3 Funcionamiento principal 13
8.4 Comparación con objetivos iniciales 13
9. Despliegue 13
9.1 Pasos para instalar o desplegar 13
9.2 Requisitos del entorno 14
9.3 Hosting o arquitectura final 14
10. Mantenimiento y mejoras futuras 14
10.1 Posibles ampliaciones 14
10.2 Mejoras pendientes 14
10.3 Escalabilidad 14
11. Conclusiones 15
11.1 Logros alcanzados 15
11.2 Dificultades encontradas 15
11.3 Aprendizajes técnicos y personales 15
12. Bibliografía 15
12.1 Libros 15
12.2 Documentación técnica 16
12.3 Artículos 16
12.4 Páginas web (formato APA) 16
13. Anexos 16
13.1 Planificación temporal 16

1. Resumen

Óptica Vision es una plataforma web desarrollada con PHP 8 y MySQL que permite a una óptica gestionar de forma centralizada a sus pacientes, el historial de revisiones oculares y la agenda de citas. La aplicación se apoya en dos perfiles diferenciados: un administrador con acceso completo a la gestión del negocio y un paciente que solo puede consultar sus propios datos en modo lectura. El proyecto resuelve un problema habitual en las ópticas pequeñas, que todavía dependen de fichas en papel u hojas de cálculo dispersas, lo que provoca pérdida de historiales, duplicados y mala experiencia para el paciente. La memoria documenta el análisis de requisitos, el modelo de datos normalizado en tercera forma normal con cuatro tablas relacionadas mediante claves foráneas con borrado en cascada, la arquitectura del sistema dividida en cinco capas lógicas, las funcionalidades implementadas para cada rol y las decisiones de seguridad adoptadas desde el inicio: sentencias preparadas con PDO contra inyección SQL, hash de contraseñas con bcrypt, regeneración del identificador de sesión tras el login, escapado HTML en toda salida dinámica para evitar XSS, validación de tipos en parámetros GET e integridad referencial garantizada por la propia base de datos. El resultado es una aplicación funcional, mantenible y desplegable en cualquier hosting compartido con soporte para PHP y MySQL.

2. Introducción

2.1 Presentación del proyecto

Óptica Vision es una aplicación web de gestión integral para una óptica pequeña o mediana. Permite registrar pacientes, mantener el historial completo de revisiones oculares (con la graduación detallada de ambos ojos) y gestionar la agenda de citas. La aplicación está pensada para ser desplegada en un servidor web estándar con PHP 8 y MySQL, sin necesidad de infraestructura adicional ni licencias de software comercial. La interfaz es totalmente web, accesible desde cualquier navegador moderno, y se adapta a dispositivos de escritorio, tablet y móvil gracias a un diseño responsive basado en CSS Grid y Flexbox.

2.2 Motivación y origen de la idea

La idea nace de cómo a la mayoría de personas y sobre todo a personas mayores les es prácticamente imposible recordar la graduación que tienen en su visión, aparte que muchas pequeñas ópticas aún gestionan a sus pacientes con agendas en papel, fichas físicas guardadas en archivadores y, en el mejor de los casos, una hoja de cálculo compartida en un ordenador del

establecimiento. Esta forma de trabajar tiene tres problemas evidentes. Primero, la información se pierde con facilidad. Segundo, no hay forma de que el paciente acceda a sus propios datos ni recuerde cuándo fue su última revisión. Y tercero, gestionar estadísticas mínimas requiere un esfuerzo manual que casi nadie se toma. Una aplicación web sencilla resuelve los tres problemas a la vez.

2.3 Contexto (problema detectado o necesidad)

El sector óptico en España está formado mayoritariamente por establecimientos pequeños e independientes. Aunque existen ERPs comerciales muy completos en el mercado, su precio mensual y su complejidad de uso los hacen poco atractivos para una óptica que atiende a pocos pacientes al día. Existe un hueco real para una solución sencilla, autoafilable y enfocada exclusivamente en lo esencial: pacientes, revisiones y citas. Óptica Vision cubre ese hueco con una aplicación web, fácil de instalar (apenas un servidor web con PHP y MySQL) y completamente gratuita.

2.4 Objetivos generales y específicos

El objetivo general del proyecto es desarrollar una plataforma web completa que permita gestionar las operaciones diarias de una óptica. Los objetivos específicos son:

1. Implementar un sistema de autenticación con dos roles diferenciados (administrador y paciente) y control de acceso por sesión.
2. Modelar la base de datos en tercera forma normal con integridad referencial garantizada por claves foráneas.
3. Construir un panel de administración con CRUD completo de pacientes, registro de revisiones y agenda de citas.
4. Ofrecer al paciente una vista de su propio historial de revisiones y datos oculares.
5. Aplicar buenas prácticas de seguridad (PDO con sentencias preparadas, hash de contraseñas, regeneración de sesión, escapado HTML) desde el inicio del desarrollo.

2.5 Metodología empleada

El desarrollo ha seguido una aproximación incremental e iterativa. En la fase inicial se elaboró el modelo de datos sobre papel y se validaron las relaciones entre tablas. Posteriormente se construyó la capa de acceso a datos (config/database.php) y la capa de autenticación (includes/auth.php), que sirven de cimiento al resto de la aplicación. A continuación se desarrolló el módulo del administrador funcionalidad por funcionalidad: pacientes, revisiones y citas. Finalmente se construyó el panel del paciente, reutilizando componentes ya creados. El control de versiones se ha llevado con Git, registrando un commit por funcionalidad terminada.

3. Análisis Inicial

3.1 Descripción del problema o necesidad

El problema concreto que resuelve la aplicación es la confusión de los pacientes a la hora de recordar su graduación o sus enfermedades de visión de cara a comunicarse con otro profesional del sector. Aparte de eso en las ópticas pequeñas hay una dispersión de información clínica y administrativa. Antes de Óptica Vision, los datos del paciente, su historial de graduaciones y sus citas se mantienen en soportes físicos diferentes (fichas, agenda en papel, libros de contabilidad) que solo podían acceder los optometristas y muchas veces podían llegar a perder. Esto dificulta consultar el historial de un paciente concreto, recordar próximas citas o compartir información entre los optometristas. La aplicación centraliza toda esa información en una única base de datos accesible desde cualquier punto del establecimiento.

3.2 Usuarios objetivos / personas involucradas

Existen dos perfiles de usuario claramente diferenciados. El primero es el administrador de la óptica: el propietario y los optometristas. Este usuario tiene acceso completo a la aplicación: puede dar de alta y baja a pacientes, registrar revisiones, gestionar la agenda de citas. El segundo perfil es el paciente. Cada paciente registrado en la óptica puede acceder con sus credenciales y ver, en modo solo lectura, sus propios datos personales, su historial de revisiones (con todas las graduaciones registradas a lo largo del tiempo) y sus citas próximas y pasadas.

3.3 Requisitos del proyecto

Los requisitos del proyecto se han agrupado en dos bloques: funcionales (lo que la aplicación debe hacer) y no funcionales (cómo debe hacerlo). Adicionalmente se han definido restricciones tecnológicas: la aplicación debe poder ejecutarse en cualquier servidor LAMP/WAMP estándar, sin dependencias propietarias ni necesidad de servicios cloud específicos, y debe funcionar correctamente con la versión 8 de PHP y la versión 8 de MySQL (o MariaDB equivalente).

3.4 Requisitos funcionales

Los requisitos funcionales identificados son:

1. El sistema debe permitir el inicio de sesión mediante email y contraseña, redirigiendo al usuario al panel correspondiente según su rol.
2. El administrador debe poder crear, editar y eliminar pacientes
3. El administrador debe poder registrar revisiones oculares con graduación detallada de ambos ojos (esfera, cilindro y eje)

4. El administrador debe poder gestionar la agenda de citas, creando nuevas citas y cambiando su estado (pendiente, confirmada, cancelada o completada).
5. El paciente debe poder consultar sus datos personales, su historial de revisiones y sus citas.
6. El sistema debe garantizar que un paciente nunca acceda a datos de otro paciente.

3.5 Requisitos no funcionales

Los requisitos no funcionales abarcan rendimiento, seguridad, usabilidad y mantenibilidad. En rendimiento, las páginas deben cargar en menos de un segundo en condiciones normales con la base de datos local. En seguridad, todas las contraseñas deben almacenarse hasheadas con bcrypt, las consultas SQL deben usar siempre sentencias preparadas para prevenir inyección, las salidas HTML deben escaparse para prevenir XSS y el identificador de sesión debe regenerarse tras el login. En usabilidad, la interfaz debe ser responsive y accesible desde cualquier navegador moderno. En mantenibilidad, el código debe estar organizado en capas claramente separadas (configuración, autenticación, vistas, lógica por rol).

3.6 Alcance y limitaciones

El alcance del proyecto cubre la gestión interna del día a día: pacientes, revisiones y citas. Quedan fuera del alcance la facturación, el control de stock de monturas y lentes, la pasarela de pagos online y la integración con sistemas externos como Hacienda o seguros médicos. Estas funcionalidades se identifican como ampliaciones futuras. Las limitaciones técnicas conocidas incluyen la ausencia de tokens CSRF (que se aplicarían en una siguiente iteración), la falta de paginación en los listados (no necesaria para los volúmenes esperados de una óptica pequeña) y la imposibilidad de que el paciente solicite citas por sí mismo en esta primera versión.

4. Estado del Arte / Tecnologías Existentes

4.1 Soluciones similares existentes

En el mercado existen varios productos comerciales orientados a la gestión de ópticas, como OptiClinic, OptisalusGes o NextOpta, que ofrecen suites completas con módulos de facturación, gestión de stock, integración con seguros médicos y conexión con bases de datos clínicas externas. También existen ERPs genéricos para consultas médicas que se adaptan al sector óptico. Estas soluciones, sin embargo, suelen requerir una cuota mensual

elevada y un proceso de instalación y formación complejo, lo que las hace poco viables para un negocio independiente y pequeño.

4.2 Comparativa de herramientas

Comparado con las soluciones comerciales, Óptica Vision sacrifica amplitud funcional a cambio de simplicidad, auto alojamiento y coste cero. Mientras que OptiClinic ofrece un módulo de facturación completo y conexión con seguros, Óptica Vision se limita a la gestión de pacientes, revisiones y citas. A cambio, no requiere cuota mensual, los datos quedan bajo el control del propietario del establecimiento y la curva de aprendizaje es prácticamente nula. Para una óptica pequeña que solo necesita un sistema centralizado de fichas y agenda, esta simplicidad es una ventaja, no un defecto.

4.3 Justificación tecnológica

La elección del stack PHP + MySQL responde a tres criterios. Primero, la disponibilidad: cualquier hosting compartido económico soporta PHP y MySQL de forma nativa, sin coste adicional. Segundo, el conocimiento: PHP y MySQL son las tecnologías centrales de nuestro ciclo formativo de DAW y dominarlas es uno de los objetivos didácticos del PFC. Y tercero, la fiabilidad: ambas tecnologías son estables, están bien documentadas y cuentan con una comunidad enorme, lo que facilita la búsqueda de soluciones a cualquier problema que pueda surgir durante el desarrollo o el mantenimiento.

4.4 Lenguajes

Los lenguajes utilizados son PHP 8 para la lógica de servidor, SQL para las consultas a la base de datos, HTML5 para la estructura semántica de las páginas, CSS3 (con variables, Grid y Flexbox) para la presentación visual y JavaScript ES6 vanilla para pequeñas mejoras de experiencia de usuario en cliente (confirmaciones, autocierre de alertas, resaltado del menú activo).

4.5 Frameworks

Conscientemente no se ha utilizado ningún framework de PHP (como Laravel, Symfony o CodeIgniter) ni de CSS (como Bootstrap o Tailwind). Esta decisión es deliberada: el objetivo del PFC es demostrar el dominio de los conceptos básicos del desarrollo web (ciclo de vida de la sesión, conexión a base de datos, separación de capas, consultas parametrizadas) sin ocultarlos detrás de la abstracción que ofrece un framework. Para un proyecto en producción a mayor escala, la migración a Laravel o Symfony sería el siguiente paso natural.

4.6 Librerías

Las únicas librerías utilizadas son las estándar incluidas en PHP (PDO para acceso a base de datos, password_hash y password_verify para gestión de contraseñas, htmlspecialchars para escapado de salida) y la API DOM nativa

del navegador para el JavaScript de cliente. No se incluyen librerías externas vía CDN ni gestores de paquetes, lo que mantiene el proyecto autocontenido y libre de dependencias.

4.7 Bases de datos

Se utiliza MySQL 8 con el motor de almacenamiento InnoDB, que soporta transacciones y, lo más importante para este proyecto, claves foráneas con borrado en cascada. El juego de caracteres es utf8mb4 con la collation utf8mb4_spanish_ci para soportar correctamente acentos y caracteres especiales en español. La base de datos está estructurada en cuatro tablas principales (usuarios, pacientes, revisiones, citas) en tercera forma normal.

4.8 Servicios cloud / hosting

El proyecto está diseñado para ejecutarse en cualquier servidor LAMP/WAMP estándar, lo que incluye prácticamente cualquier hosting compartido del mercado. Para el desarrollo se ha utilizado XAMPP en local, que empaqueta Apache, PHP y MySQL en un único instalador. Para el despliegue en producción se contempla un hosting compartido económico (000webhost, Heliohost, hosting de 1and1 o similares) o un VPS básico con Apache y certificado Let's Encrypt para HTTPS.

4.9 Hardware / infraestructura (si aplica)

El proyecto no requiere hardware específico más allá de un servidor con conexión a internet. Para el desarrollo se ha utilizado un equipo personal de gama media con sistema operativo Windows 11, sobre el que se ha instalado XAMPP. La aplicación es totalmente compatible con Linux y macOS, ya que no usa ninguna característica específica del sistema operativo. En producción, los requisitos mínimos son los típicos de un hosting compartido: 100 MB de espacio en disco, 50 MB de base de datos y soporte para PHP 8 y MySQL 5.7 o superior.

5. Diseño del Proyecto

5.1 Diseño funcional

Casos de uso

Los casos de uso principales son:

1. Iniciar sesión
2. Cerrar sesión.
3. Crear pacientes (admin).
4. Editar paciente (admin), CU05 Eliminar paciente (admin).

5. Listar pacientes (admin).
6. Registrar revisión (admin).
7. Listar y filtrar revisiones (admin).
8. Crear cita (admin).
9. Cambiar estado de cita (admin).
10. Eliminar cita (admin).
11. Consultar panel personal (pacientes).
12. Consultar historial de revisiones (pacientes).
13. Consultar citas (pacientes).

5.2 Diseño técnico

Arquitectura del sistema

La arquitectura es de servidor monolítico con renderizado del lado del servidor. El navegador del cliente realiza una petición HTTP a Apache, que invoca al intérprete de PHP. PHP ejecuta el fichero solicitado, que normalmente carga la configuración de base de datos, valida la sesión, ejecuta consultas con PDO y renderiza HTML mediante plantillas (header.php + contenido + footer.php). El HTML resultante se devuelve al navegador, que lo muestra y aplica los estilos de assets/css/style.css y los comportamientos de assets/js/main.js. No hay API REST ni comunicación AJAX: todo el flujo es síncrono.

Modelo entidad–relación

El modelo entidad-relación tiene cuatro entidades principales. Usuario es la entidad de autenticación, con id, nombre, apellidos, email (único), password (hash) y rol. Paciente extiende a Usuario en una relación 1:1 con datos clínicos: dni (único), fecha_nacimiento, teléfono y dirección. Revisión es una entidad débil dependiente de Paciente, con la fecha, el optometrista, la graduación detallada de ambos ojos y campos de diagnóstico y observaciones. Cita también es débil respecto a Paciente, con fecha y hora, motivo y un estado (pendiente, confirmada, cancelada o completada). Las relaciones se implementan con claves foráneas con borrado en cascada.

Diseño lógico de la base de datos

El diseño lógico se materializa en cuatro tablas. La tabla usuarios tiene como clave primaria un id autoincremental y una clave única en email. La tabla pacientes tiene id propio, una clave foránea usuario_id (con UNIQUE para imponer la relación 1:1) y una clave única en dni. La tabla revisiones tiene id, una clave foránea paciente_id y diez columnas para los datos clínicos. La tabla

citas tiene id, paciente_id, fecha_hora y un campo estado de tipo ENUM. Todas las claves foráneas están definidas con ON DELETE CASCADE para garantizar la integridad referencial.

Diseño de interfaz (mockups, wireframes)

La interfaz sigue un diseño minimalista y profesional. La paleta se construye sobre dos colores principales (azul marino #1a2e5a como primario y azul medio #4a90d9 como acento) más los colores semánticos habituales (verde para éxito, rojo para peligro, amarillo para advertencia). La tipografía es Segoe UI como fuente principal, con caídas a system-ui y sans-serif. Cada página tiene una estructura común: cabecera fija con logo y menú de navegación según rol, contenido principal centrado y pie de página discreto. Los formularios se estructuran en grids de dos columnas que colapsan a una en el móvil.

6. Desarrollo / Implementación

6.1 Entorno de desarrollo

El entorno de desarrollo se ha montado sobre XAMPP para Windows, que incluye Apache 2.4, PHP 8 y MariaDB 10. Como editor se ha utilizado Visual Studio Code con las extensiones PHP Intelephense (autocompletado y análisis estático) y SQLTools (acceso a la base de datos desde el editor). El control de versiones se ha llevado con Git, con un repositorio local más una copia remota de respaldo. Las pruebas se han realizado en Chrome, Firefox y Edge para garantizar compatibilidad cross-browser.

6.2 Configuraciones importantes

Las configuraciones más relevantes están centralizadas en config/database.php, donde se definen las constantes DB_HOST, DB_NAME, DB_USER, DB_PASS y DB_CHARSET. La conexión PDO se configura con los atributos ATTR_ERRMODE en EXCEPTION (para que los errores SQL lancen excepciones), ATTR_DEFAULT_FETCH_MODE en FETCH_ASSOC (para devolver los resultados como arrays asociativos) y ATTR_EMULATE_PREPARES en false (para usar sentencias preparadas reales del servidor MySQL en lugar de emularlas en cliente).

6.3 Desarrollo de módulos

El desarrollo se ha estructurado por módulos funcionales. El módulo de autenticación (login.php, logout.php, includes/auth.php) se desarrolló en primer lugar. Después se construyó el módulo de pacientes (admin/pacientes.php), seguido del módulo de revisiones (admin/nueva_revision.php y admin/revisiones.php) y finalmente el módulo de citas (admin/citas.php). En paralelo se desarrolló el módulo del paciente (paciente/dashboard.php, paciente/revisiones.php, paciente/citas.php), que reutiliza la misma capa de datos y los mismos componentes de presentación.

6.4 Integraciones externas

El proyecto no incluye integraciones con servicios externos en esta primera versión. Se ha valorado integrar el envío de correos electrónicos para notificar al paciente la confirmación de citas (utilizando una librería como PHPMailer y un servicio como Gmail SMTP), pero finalmente se ha pospuesto a una siguiente iteración para mantener el alcance acotado. Tampoco se ha integrado pasarela de pagos online ni firma digital de informes de revisión, ambos identificados como ampliaciones futuras.

6.5 Gestión del código y control de versiones (Git)

El control de versiones se ha realizado con Git. La estructura de ramas es sencilla: una rama main que contiene la versión estable y ramas de funcionalidad cuando se aborda algo importante. Cada commit corresponde a una funcionalidad completa o a una corrección concreta, con mensajes descriptivos siguiendo el estilo imperativo ("añade alta de pacientes", "corrige hash de contraseña en edición"). El historial completo queda como evidencia del proceso de desarrollo.

6.6 Pruebas realizadas

Las pruebas realizadas se dividen en tres categorías: pruebas manuales de cada caso de uso ejecutadas tras implementar cada funcionalidad; pruebas de integración del flujo completo (login, navegación, creación de datos, cierre de sesión, reapertura) ejecutadas al final de cada módulo; y pruebas de compatibilidad en distintos navegadores y resoluciones. No se han implementado pruebas automatizadas con PHPUnit en esta primera versión, pero se identifica como una mejora a incorporar antes de un despliegue productivo real.

6.7 Pruebas unitarias

Las pruebas unitarias automatizadas no forman parte del alcance de esta primera versión, pero se ha definido qué funciones serían las primeras candidatas a tener cobertura: las funciones de includes/auth.php (requireLogin, requireAdmin, requirePaciente, isLoggedIn, isAdmin), la función getDB de config/database.php y, en una capa intermedia que aún no existe pero que se extraería en una refactorización, las funciones de validación y persistencia de pacientes, revisiones y citas. La herramienta candidata es PHPUnit.

6.8 Pruebas de integración

Las pruebas de integración se han ejecutado de forma manual, siguiendo guiones predefinidos que recorren los flujos completos de la aplicación: "el administrador da de alta un paciente, le registra una revisión, le crea una cita, cierra sesión, el paciente entra y verifica que ve sus datos". Cada flujo se ha ejecutado al final de cada módulo y antes de cada commit importante. La automatización con Selenium o Playwright queda identificada como mejora futura.

6.9 Rendimiento

El rendimiento de la aplicación se considera muy adecuado para los volúmenes esperados. Las páginas más pesadas (el dashboard del administrador, que ejecuta cuatro consultas COUNT y dos SELECT con JOIN) renderizan en menos de 50 milisegundos en local con la base de datos de prueba. Para los volúmenes esperados de una óptica pequeña (hasta unos cientos de pacientes y miles de revisiones), no se anticipan problemas de rendimiento. En caso de crecimiento significativo, las primeras optimizaciones serían añadir índices a paciente_id en revisiones y citas y paginar los listados.

7. Seguridad

7.1 Identificación de riesgos

Los riesgos de seguridad principales identificados son los típicos del OWASP Top 10 aplicados a una aplicación PHP/MySQL: inyección SQL en formularios y parámetros GET, Cross-Site Scripting (XSS) reflejado en la salida de datos del usuario, Cross-Site Request Forgery (CSRF) en operaciones POST, filtración de contraseñas en caso de compromiso de la base de datos, session fixation y

manipulación de URL para acceder a recursos de otro usuario. Cada uno de estos riesgos tiene una mitigación concreta documentada en los apartados siguientes.

7.2 Autenticación y autorización

La autenticación se gestiona con email y contraseña. Las contraseñas se almacenan hasheadas con bcrypt mediante password_hash y se verifican con password_verify, ambas funciones nativas de PHP. La autorización se gestiona en dos capas: a nivel de servidor cada página privada llama a requireAdmin() o requirePaciente() al inicio, redirigiendo al usuario si no tiene los permisos adecuados; y a nivel de UI, el menú de navegación se renderiza dinámicamente según el rol del usuario en sesión.

7.3 Gestión de sesiones

Las sesiones se gestionan con la API nativa de PHP. Tras un login correcto se invoca session_regenerate_id(true) para mitigar ataques de session fixation. En el cierre de sesión se vacía el array $_SESSION, se elimina la cookie de sesión explícitamente y se llama a session_destroy. La cookie de sesión es del tipo "de sesión" (sin caducidad explícita), por lo que se elimina automáticamente al cerrar el navegador. En una versión productiva con HTTPS se añadirían los flags Secure y HttpOnly a la cookie.

7.4 Cifrado de datos

El cifrado se aplica en dos puntos. Las contraseñas se almacenan hasheadas con bcrypt (no es cifrado reversible, sino una función hash de un solo sentido con salt incorporado), de modo que aunque la base de datos se vea comprometida, las contraseñas no son recuperables en texto plano. En tránsito, en producción la aplicación se servirá exclusivamente sobre HTTPS con un certificado Let’s Encrypt, lo que garantiza que las credenciales y los datos clínicos viajan cifrados entre el navegador y el servidor.

8. Resultados

8.1 Presentación del producto final

El producto final es una aplicación web completamente funcional que cumple los objetivos planteados en el análisis. Tras instalar XAMPP e importar el script

SQL, el sistema arranca y permite al administrador iniciar sesión con las credenciales de prueba (admin@optica.local /password). Desde ese momento puede gestionar pacientes, registrar revisiones y administrar la agenda. Los pacientes precargados pueden iniciar sesión con sus respectivas credenciales y ver su historial. Toda la navegación es responsive y funciona correctamente en móvil y tablet.

8.2 Capturas de pantalla o diagramas finales

[En este apartado se incluirán las capturas de pantalla definitivas del producto final: pantalla de inicio de sesión, panel del administrador con sus indicadores, listado de pacientes con el formulario de alta, formulario de nueva revisión con la tabla de graduación, listado de revisiones, agenda de citas con los badges de estado, panel del paciente y vista de "Mis Revisiones". Se complementan con los diagramas finales de arquitectura, modelo entidad-relación y flujo de autenticación generados durante la fase de diseño.]

8.3 Funcionamiento principal

El funcionamiento principal de la aplicación se articula alrededor de la sesión PHP. Una vez autenticado, el usuario navega entre las distintas páginas del rol que le corresponda. Cada acción que modifica datos (alta, edición, borrado, cambio de estado de cita) se realiza con un POST o un GET con parámetros validados, ejecuta una sentencia preparada contra la base de datos y, tras el commit, redirige a la página anterior con un mensaje de éxito o de error. El paciente, que no tiene operaciones de escritura, simplemente consulta sus datos en modo lectura.

8.4 Comparación con objetivos iniciales

Comparando con los objetivos planteados en el apartado 2.4, se han cumplido los cinco objetivos específicos:

1. El sistema de autenticación con dos roles está implementado y funcional.
2. La base de datos está normalizada en tercera forma normal con cuatro tablas e integridad referencial garantizada por claves foráneas.
3. El panel de administración tiene CRUD completo de pacientes, registro de revisiones y agenda de citas.
4. El paciente accede a su historial en modo lectura.
5. Las buenas prácticas de seguridad están aplicadas (PDO preparado, bcrypt, session_regenerate_id, htmlspecialchars, validación de tipos).

9. Despliegue

9.1 Pasos para instalar o desplegar

El despliegue requiere cuatro pasos:

1. Copiar la carpeta Optica-web/ al directorio raíz del servidor web (htdocs en XAMPP, public_html en muchos hostings compartidos).
2. Crear una base de datos MySQL llamada optica_db e importar el fichero sql/optica.sql, que crea las cuatro tablas y carga los datos de prueba.
3. Editar config/database.php para ajustar los datos de conexión (host, usuario y contraseña) si difieren del entorno por defecto.
4. Acceder a la aplicación a través del navegador en la URL del servidor. La pantalla de login debe aparecer correctamente.

9.2 Requisitos del entorno

Los requisitos del entorno de producción son: PHP 8.0 o superior con las extensiones PDO y pdo_mysql habilitadas; MySQL 5.7 o superior, o MariaDB equivalente; un servidor web (Apache, Nginx o el servidor incorporado de PHP) capaz de ejecutar PHP; al menos 50 MB de espacio en disco para los archivos del proyecto y 50 MB adicionales para la base de datos; y, para producción, certificado HTTPS válido (puede obtenerse gratis de Let's Encrypt). No se requieren extensiones PHP adicionales más allá de las incluidas en la instalación estándar.

9.3 Hosting o arquitectura final

La arquitectura de producción contemplada es la siguiente: un único servidor con Apache + PHP + MySQL alojado en un hosting compartido económico o un VPS pequeño. La base de datos vive en el mismo servidor que la aplicación, sin necesidad de servidor de base de datos separado. El acceso es exclusivamente a través de HTTPS con certificado Let's Encrypt renovado automáticamente. Los backups de la base de datos se programarían como tarea cron diaria con mysqldump, almacenando los volcados durante 30 días.

10. Mantenimiento y mejoras futuras

10.1 Posibles ampliaciones

Las ampliaciones identificadas como más interesantes son:

- La solicitud de citas por parte del paciente con validación posterior del administrador, lo que cambia significativamente la propuesta de valor.
- La exportación a PDF del informe de revisión, útil para que el paciente pueda llevarlo a otra consulta.
- La integración con un sistema de envío de correos (PHPMailer + SMTP) para notificar confirmaciones y recordatorios automáticos.
- Un módulo de estadísticas avanzadas con gráficos sobre tipos de graduaciones más frecuentes, evolución del número de pacientes y carga de trabajo por día de la semana.

10.2 Mejoras pendientes

Entre las mejoras pendientes que afectan a la calidad del código y la seguridad, las más prioritarias son:

- Añadir tokens CSRF a todos los formularios POST para mitigar el riesgo de Cross-Site Request Forgery.
- Forzar el cambio de contraseña en el primer acceso del paciente, que actualmente se anuncia en la UI pero no está implementado;
- Añadir paginación a los listados de revisiones y citas en preparación para volúmenes mayores.
- Introducir tests automatizados con PHPUnit, al menos para las funciones de auth.php y los casos de uso críticos.

10.3 Escalabilidad

La escalabilidad es adecuada para el caso de uso planteado (una óptica pequeña con pocos cientos de pacientes). Para volúmenes mayores las estrategias serían: añadir índices secundarios a las columnas de búsqueda más frecuentes (paciente_id en revisiones y citas, fecha_hora en citas); paginar los listados; cachear con APCu las consultas de solo lectura más frecuentes; y, en el extremo, separar la base de datos a un servidor dedicado y poner un balanceador delante de varias instancias de Apache.

11. Conclusiones

11.1 Logros alcanzados

Los logros principales del proyecto son: una aplicación completamente funcional que cumple los cinco objetivos específicos planteados; un modelo de datos normalizado y con integridad referencial garantizada por la propia base de datos; una capa de acceso a datos centralizada con PDO; buenas prácticas de seguridad aplicadas desde la primera línea de código (sentencias preparadas, hash de contraseñas, regeneración de sesión, escapado HTML); un diseño responsive construido con CSS variables, Grid y Flexbox sin recurrir a frameworks externos; y un código modular y mantenible organizado en capas claramente diferenciadas.

11.2 Dificultades encontradas

Las dificultades principales han estado en dos áreas. La primera, el control de acceso por rol: garantizar que cada página privada llama a la función de autorización correcta y que un paciente nunca puede acceder a datos de otro, ni siquiera manipulando la URL, ha requerido revisar cuidadosamente cada controlador. La segunda, el modelado del formulario de revisión ocular: representar de forma clara la graduación de los dos ojos con sus tres parámetros (esfera, cilindro, eje) y validar los rangos correctos ha requerido varias iteraciones hasta llegar a una interfaz natural.

11.3 Aprendizajes técnicos y personales

Los aprendizajes técnicos más valiosos han sido:

- La importancia de aplicar las buenas prácticas de seguridad desde el primer commit, no como una capa que se añade al final.
- El valor de centralizar la conexión a base de datos y la autorización en archivos comunes para reducir duplicación.
- La utilidad de las claves foráneas con borrado en cascada para mantener la integridad de los datos sin esfuerzo.

En el plano personal, el proyecto ha consolidado la confianza en un stack que conozco bien y ha demostrado que la sencillez es a menudo la mejor herramienta de ingeniería.

12. Bibliografía

12.1 Libros

- Temario Desarrollo de web en entorno de servidor
- Temario Desarrollo de web en entorno de cliente
- Temario Diseño de interfaces

12.2 Documentación técnica

- Documentación oficial de PHP https://www.php.net/manual/es/
- Documentación oficial de MySQL 8 https://dev.mysql.com/doc/refman/8.0/en/
- MDN Web Docs https://developer.mozilla.org/
- DOM en JavaScript. OWASP Top 10 https://owasp.org/www-project-top-ten/

12.4 Páginas web

- PHP Group https://www.php.net/
- PHP password_hash https://www.php.net/manual/en/function.password-hash.php
- PHP Password Hashing https://www.php.net/manual/en/faq.passwords.php
- XAMPP Apache + MariaDB + PHP + Perl https://www.apachefriends.org/
- PHP The Right Way https://phptherightway.com
- CSS-Tricks https://css-tricks.com

13. Anexos

13.1 Planificación temporal

La planificación temporal del proyecto se ha distribuido en seis semanas. La primera semana se dedicó al análisis de requisitos y al diseño del modelo de datos. La segunda y tercera al desarrollo del módulo de autenticación y de la gestión de pacientes. La cuarta al módulo de revisiones, incluyendo el diseño cuidadoso del formulario de graduación ocular. La quinta al módulo de citas y al panel del paciente. Y la sexta a las pruebas integradas, la preparación de la defensa y la grabación del vídeo.